Cookies and Sessions

(https://github.com/learn-co-curriculum/python-p4-login-sessions-cookies) (https://github.com/learn-co-curriculum/python-p4-login-sessions-cookies/issues/new)

Learning Goals

- Explain what a cookie is and what cookies can be used for.
- Identify how cookies are part of the request/response cycle.
- Explain what a session is in Flask.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

Cookies are small pieces of information that are sent from the server to the client. They are then stored on the client (in the browser) and sent back to the server with each subsequent request.

HTTP is a **stateless** protocol, since the server doesn't maintain information about each client for all requests. Cookies help make **stateful** HTTP requests by providing a mechanism for sending additional information to the server with each request.

Cookies are *domain-specific*. The browser stores cookies for each domain (e.g. `nytimes.com`) separately, and only cookies for that domain are sent back to the server with subsequent requests.

Cookies are typically used to store session information (user login, shopping cart, etc.), personalization (user preferences, themes, etc.) and tracking information (analyzing user behavior). They provide a way for us to verify who a user is once, and then remember it for their entire session. Without cookies, you would have to provide your username and password on every single request to the server.

In this document, we'll cover what cookies are, how they fit into the HTTP request/response cycle, and how you can access them within your Flask application.

Page View Tracking

Let's say we want to build a paid blog site, like Medium. Users can view up to 5 articles per month for free, but after that, they need to subscribe to see more content.

How can we keep track of a user's page views?

We could make a `User` model, and create a `pageviews_remaining` attribute to keep track of how many articles the user can read; but ideally, we'd like to let users browse the site without needing to log in, and still have some way of tracking their page views.

When a user views an article, their browser will make a request to `/articles/<int:id>`.

Remember what's included in an HTTP request:

- An **HTTP verb (https://www.rfc-editor.org/rfc/rfc9110.html)**, like `GET`, `PATCH`, or `POST`.
- A path (`/articles/<int:id>`).
- Various headers containing additional metadata (like the `Content-Type` of the request).

HTTP servers are typically stateless. They receive requests, process them, return data, then forget about them. This means that all required information must be passed with the request, either in the path or in the headers.

For example, `GET` requests usually encode the necessary information in the path. When you write a route matching `/articles/<int:id>`, you are telling Flask to pull the value `id` from the request path and save it in an `id` scoped to the view function. In your app, you'll probably have a

view that looks something like:

```
@app.route('/articles/<int:id>', methods=['GET'])
def show_article(id):
  article = Article.query.filter(Article.id === id).first()
  return jsonify(article.to_dict(), 200)
```

This code loads the row for that article from the database and returns it as a SQLAlchemy model object, which is then serialized as JSON. All the information needed for the `GET` request — in this case, the `id` of the article to render — is included in the path.

Similarly, if we want to be able to keep track of page views, we need to figure out how to include that information in the request, either in the path or the headers.

It would be possible, though quite convoluted, to store this information in the path. Our JavaScript application could keep track of the articles the user has viewed, and include the remaining page views as an additional query parameter in the request: `/articles/3?pageviews_remaining=5`. However, there are a few flaws to this approach: most obviously, it would be incredibly simple for the user to change this number in the request and circumvent our paywall: `/articles/3?pageviews_remaining=999`.

Luckily, cookies allow us to store this information in the only other place available to us: HTTP headers.

What's a Cookie, Anyway?

Let's see what **the spec (http://tools.ietf.org/html/rfc6265)** has to say:

```
This section outlines a way for an origin server to send state
information to a user agent and for the user agent to return the
state information to the origin server.

To store state, the origin server includes a Set-Cookie header in an
HTTP response.  In subsequent requests, the user agent returns a
Cookie request header to the origin server.  The Cookie header
```

```
contains cookies the user agent received in previous Set-Cookie
headers.  The origin server is free to ignore the Cookie header or
use its contents for an application-defined purpose.
```

The description is quite technical, so let's look at their example:

```
== Server -> User Agent ==
Set-Cookie: SID=31d4d96e407aad42

== User Agent -> Server ==
Cookie: SID=31d4d96e407aad42
```

In this example, the server is an HTTP server, and the User Agent is a browser. The server responds to a request with the `Set-Cookie` header. This header sets the value of the `SID` cookie to `31d4d96e407aad42`.

Next, when the user visits another page on the same server, the browser sends the cookie back to the server, including the `Cookie: SID=31d4d96e407aad42` header in its request.

Cookies are stored in the browser. The browser doesn't care about what's in the cookies you set. It just stores the data and sends it along on future requests to your server. You can think of them as a hash— and indeed, as we'll see later, Flask exposes cookies with a method that behaves much like a hash.

Using Cookies

So how would we use a cookie to store a reference to the user's page views? Let's say that we create a page view counter the first time a user views an article. Then, in the **response**, we might include the header:

```
== Server -> User Agent ==
Set-Cookie: pageviews_remaining=5
```

When the user views another article, we can instruct the browser to include the cookie in the **request** headers:

```
== User Agent -> Server ==
Cookie: pageviews_remaining=5
```

We can look at this HTTP header, get the `pageviews_remaining` from it, and write some conditional logic to customize the response based on the `pageviews_remaining` to either return the article, or return a message indicating that our frontend should show the paywall.

Security Concerns

Cookies are stored as plain text in a user's browser. Therefore, the user can see what's in them, and they can set them to anything they want.

If you open the developer console in your browser, you can see the cookies set by the current site. In Chrome's console, you can find this under `Application > Cookies`. You can delete any cookie you like. For example, if you delete your `user_session` cookie on `github.com` and refresh the page, you will find that you've been logged out.

You can also edit cookies, for example with **this extension (https://chrome.google.com/webstore/detail/editthiscookie/fngmhnnpilhplaeedifhccceomclgfbg?hl=en)**.

This presents a problem for us. If users can edit their `pageviews_remaining` cookie, then they can easily give themselves an unlimited amount of page views.

Flask Never Fails!

Fortunately, Flask has a solution to this. Instead of sending our cookies in plain text, we can use Flask to **encrypt** and **sign** a special cookie known as a session using the `session` module. The `session` module is imported from `flask`, and it behaves like a dictionary:

```
session['pageviews_remaining'] = 5
```

You can store any simple Python object in the session.

By default, Flask manages all session data in a single cookie. It *serializes* all the key/value pairs you set with `session`, converting them from a Python object into a big string. Whenever you set a key with the `session` module, Python updates the value of its session cookie to this big string.

Furthermore, `session` is a part of the application's *request context*. This means that we don't have to create it manually or pass it as an argument to any of our views- it's accessible whenever the `request` is!

When you set cookies this way, Flask **signs** them to prevent users from tampering with them. Your Flask server has a key, configured in your app:

```
app.secret_key = 'BAD_SECRET_KEY'
```

The best strategy for generating a good secret key is using the `os` module from the command line:

```
$ python -c 'import os; print(os.urandom(16))'
# => b'_5#y2L"F4Q8z\n\xec]/'

# keep this somewhere safe!
```

It's guaranteed that given the same message and key, Flask will produce the same output. Also, without the key, it is practically impossible to know what Flask would return for a given message. That is, signatures can't be forged.

Flask creates a signature for every session cookie it sets, and appends the signature to the cookie.

When it receives a cookie, Flask verifies that the signature matches the content.

This prevents cookie tampering. If a user tries to edit their cookie and change the `pageviews_remaining`, the signature won't match, and Flask will silently ignore the cookie and set a new one.

Cryptography is a deep rabbit hole. At this point, you don't need to understand the specifics of how cryptography works, just that Flask and other frameworks use it to ensure that session data which is set on the server can't be edited by users.

Conclusion

Cookies are foundational for the modern web.

Most sites use cookies, to let their users log in, keep track of their shopping carts, or record other ephemeral session data. Almost nobody thinks these are bad uses of cookies: nobody really believes that you should have to type in your username and password on every page, or that your shopping cart should clear if you reload the page.

But cookies let you store data in a user's browser, so by nature, they can be used for more controversial endeavors.

For example, Google AdWords sets a cookie and uses that cookie to track what ads you've seen and which ones you've clicked on. The tracking information helps AdWords decide what ads to show you.

This is why, if you click on an ad, you may find that the ad follows you around the internet. It turns out that this behavior is as effective as it is annoying: people are far more likely to buy things from ads that they've clicked on once.

This use of cookies worries people and the EU **has created legislation around the use of cookies (https://en.wikipedia.org/wiki/General_Data_Protection_Regulation)** .

Cookies, like any technology, are a tool. In the rest of this module, we're going to be using them to let users log in. Whether you later want to use them in such a way that the EU passes another law is up to you.

Check For Understanding

Before you move on, make sure you can answer the following questions:

▶ *1. What do we mean when we say that HTTP is stateless?*

▶ *2. What attribute do we need to set to ensure that data is encrypted in Flask sessions?*

Resources

- Introduction to Identity and Access Management (IAM) - auth0 (https://auth0.com/docs/get-started/identity-fundamentals/identity-and-access-management)
- HTTP RFC Section 9 — Methods (https://www.rfc-editor.org/rfc/rfc9110.html)
- RFC 6265 — HTTP State Management Mechanism (the cookie spec) (http://tools.ietf.org/html/rfc6265)
- Session data in Python Flask - pythonbasics.org (https://pythonbasics.org/flask-sessions/)
- General Data Protection Regulation (https://en.wikipedia.org/wiki/General_Data_Protection_Regulation)
- Legal Cookie Requirements (https://termly.io/resources/templates/cookie-policy-template/#are-you-legally-required-to-have-a-cookie-policy)